In [ ]:
```
import marimo as mo
import z3
from math import isqrt
from time import perf_counter
```

TP2 — Exercício 3: Multiplicação de Inteiros

1. Introdução e requisitos

Pretende-se modelar e verificar formalmente o programa de multiplicação do CFA (*Control Flow Automaton*) fornecido na unidade curricular. Utilizamos **Z3** para construir um *Safe First Order Transition System* (SFOTS) e verificar as propriedades pedidas. O notebook segue a ordem das quatro alíneas:

Requisito	Objetivo
3.a	Definir variáveis de estado, predicado inicial, transições e estados de erro.
3.b	Verificar por k-indução se $xy + z = ab$ é um invariante.
3.c	Restringir inicialmente $a, b < 2^{n/2}$ e provar a inacessibilidade de erros.
3.d	Especificar e provar a terminação para todos os operandos iniciais.

O enunciado usa precisão finita de n bits e um CFA com **sete nodos** e **oito ligações**. Não introduzimos outros nodos ou autotransições.

Convenção aritmética

Os valores dos BitVecs são interpretados **sem sinal**. A escolha é explícita: o documento de apoio descreve também o overflow com sinal numa ALU. Aqui consideramos overflow quando o resultado exato de uma soma ou duplicação não cabe no intervalo $0, \ldots, 2^n - 1$. Esta convenção afeta os resultados de segurança e deve ser confirmada na defesa como uma decisão de modelação.

Matéria utilizada

A formulação usa conceitos do **Capítulo 3** (SMT, k-indução e *lookahead*) e do **Capítulo 4** (sistemas de transição e segurança), necessários porque são expressamente pedidos neste exercício. Restringindo-nos aos Capítulos 1 e 2, seria possível verificar apenas instâncias ou percursos finitos, não substituir estas demonstrações gerais sobre estados alcançáveis.

Contribuição da LLM: organização dos requisitos e discussão das escolhas aritméticas e da estratégia de prova.

2. Formulação do SFOTS — alínea 3.a

2.1. Variáveis e estados

Seja $n \geq 1$ a largura do modelo; os testes principais utilizam $n = 8$. O estado é um tuplo:

$$S = (pc, a, b, x, y, z)$$

- pc é o nodo atual: `init`, `skip`, `left`, `right`, `stop`, `error_L` ou `error_R`;
- a, b são os operandos originais, preservados em todas as transições;
- x é o multiplicando que pode ser duplicado;
- y é a parte do multiplicador ainda não processada;
- z é o acumulador do produto parcial.

Todas as variáveis aritméticas têm o tipo BitVec de largura n. Para uma transição utilizamos o estado seguinte $S' = (pc', a', b', x', y', z')$.

O SFOTS é o quádruplo $\Sigma = \langle X, I, T, E \rangle$, onde:

$$I(S) \equiv (pc = init)$$

$$E(S) \equiv (pc = error_L) \vee (pc = error_R)$$

Antes da ação `init` os registos x, y, z podem ter valores arbitrários. É a primeira transição que os inicializa. Este detalhe é decisivo para a interpretação do invariante da alínea 3.b.

2.2. Relação de transição e overflow

A relação global é a disjunção das **oito ligações** do CFA:

$$T(S, S') = \bigvee_{i=1}^{8} T_i(S, S').$$

Ligação	Guarda	Ação sobre (x, y, z)
`init → skip`	`True`	$(a, b, 0)$
`skip → left`	$y \neq 0 \wedge even(y)$	Manter
`skip → right`	$y \neq 0 \wedge odd(y)$	Manter
`skip → stop`	$y = 0$	Manter

Ligação	Guarda	Ação sobre (x, y, z)
`left → skip`	$\neg OF_L$	$(2x, \lfloor y/2 \rfloor, z)$
`left → error_L`	OF_L	Mesma ação
`right → skip`	$\neg OF_R$	$(x, y-1, z+x)$
`right → error_R`	OF_R	Mesma ação

Em todas as ligações $a' = a$ e $b' = b$. O ramo para erro tem precedência: a guarda para `skip` inclui explicitamente a negação do overflow.

O overflow é calculado **antes de truncar** o resultado para n bits:

$$OF_L \equiv 2\widehat{x} \geq 2^n, \qquad OF_R \equiv \hat{z} + \widehat{x} \geq 2^n.$$

As variáveis são alargadas a $n+1$ bits com `ZeroExt` e a divisão de y por dois é realizada com o deslocamento lógico `LShR`. As operações guardadas em x' e z' continuam a ser modulares.

Decisão de implementação: utilizar uma função parametrizada `construir_sfots(n)`, tanto nas provas principais com $n = 8$ como na experiência de desempenho. Assim não existem dois modelos independentes do CFA.

In [ ]:
```python
# Contribuição LLM: função parametrizada que constrói o SFOTS e as propriedades.
Local, (INIT, SKIP, LEFT, RIGHT, STOP, ERROR_L, ERROR_R) = z3.EnumSort(
    "LC_locais_minimo", [
        "init", "skip", "left", "right", "stop", "error_L", "error_R"
    ]
)

def construir_sfots(n):
    """Modelo do CFA para n bits, sem alterar o autómato do enunciado."""
    if type(n) is not int or n < 1:
        raise ValueError("A largura n tem de ser um inteiro positivo.")

    def estado(nome):
        return (
            z3.Const(f"pc_{nome}_{n}", Local),
            *(z3.BitVec(f"{v}_{nome}_{n}", n) for v in ("a", "b", "x", "y", "z"))
        )

    S, S1, S2 = estado("s"), estado("s1"), estado("s2")
    pc, a, b, x, y, z = S
    pc1, a1, b1, x1, y1, z1 = S1
    zero, um = z3.BitVecVal(0, n), z3.BitVecVal(1, n)
    even = (y & um) == zero
    odd = (y & um) == um
```

```
# Extensão antes da operação: o overflow não fica ocultado pelo módulo.
dobro = z3.ZeroExt(1, x) << 1
soma = z3.ZeroExt(1, z) + z3.ZeroExt(1, x)
capacidade = z3.BitVecVal(2**n, n + 1)
of_l, of_r = z3.UGE(dobro, capacidade), z3.UGE(soma, capacidade)

manter = z3.And(x1 == x, y1 == y, z1 == z)
iniciar = z3.And(x1 == a, y1 == b, z1 == zero)
acao_left = z3.And(
    x1 == z3.Extract(n - 1, 0, dobro),
    y1 == z3.LShR(y, 1), z1 == z
)
acao_right = z3.And(
    x1 == x, y1 == y - um,
    z1 == z3.Extract(n - 1, 0, soma)
)

def ligacao(origem, destino, guarda, acao):
    return z3.And(
        pc == origem, pc1 == destino, guarda, acao,
        a1 == a, b1 == b
    )

T = z3.Or(
    ligacao(INIT, SKIP, True, iniciar),
    ligacao(SKIP, LEFT, z3.And(y != zero, even), manter),
    ligacao(SKIP, RIGHT, z3.And(y != zero, odd), manter),
    ligacao(SKIP, STOP, y == zero, manter),
    ligacao(LEFT, SKIP, z3.Not(of_l), acao_left),
    ligacao(LEFT, ERROR_L, of_l, acao_left),
    ligacao(RIGHT, SKIP, z3.Not(of_r), acao_right),
    ligacao(RIGHT, ERROR_R, of_r, acao_right)
)

def erro(s):
    return z3.Or(s[0] == ERROR_L, s[0] == ERROR_R)

def terminal(s):
    return z3.Or(s[0] == STOP, erro(s))

def controlo(s):
    p, _, _, _, yy, _ = s
    return z3.And(
        z3.Implies(p == LEFT, z3.And(yy != 0, (yy & um) == 0)),
        z3.Implies(p == RIGHT, z3.And(yy != 0, (yy & um) == um))
    )

def produto_modular(s):
    _, aa, bb, xx, yy, zz = s
    return xx * yy + zz == aa * bb

def produto_exato(s):
    _, aa, bb, xx, yy, zz = s
    ext = lambda v: z3.ZeroExt(n, v)
    return ext(xx) * ext(yy) + ext(zz) == ext(aa) * ext(bb)
```

```
    I = pc == INIT
    E = erro(S)
    J = lambda s: z3.And(
        z3.Or(s[0] == INIT, produto_modular(s)), controlo(s)
    )

    # Limite inteiro equivalente a a,b < 2^(n/2), mesmo para n ímpar.
    limite = z3.BitVecVal(isqrt(2**n - 1), n)
    def limites(s):
        return z3.And(z3.ULE(s[1], limite), z3.ULE(s[2], limite))

    I_c = z3.And(I, limites(S))
    def J_c(s):
        return z3.And(
            limites(s), z3.Or(s[0] == INIT, produto_exato(s)),
            controlo(s), z3.Not(erro(s))
        )

    # Função de progresso em n+2 bits, sem overflow da função R.
    def R(s):
        p, _, bb, _, yy, _ = s
        y_w, b_w = z3.ZeroExt(2, yy), z3.ZeroExt(2, bb)
        return z3.If(p == INIT, 2 * b_w + 2,
            z3.If(p == SKIP, 2 * y_w + 1,
                z3.If(z3.Or(p == LEFT, p == RIGHT), 2 * y_w,
                      z3.BitVecVal(0, n + 2))))

    def lookahead(s, t):
        return z3.Or(
            z3.And(terminal(s), terminal(t)),
            z3.And(z3.Not(terminal(s)), controlo(s), controlo(t),
                   z3.ULT(R(t), R(s)))
        )

    # Em cada local não terminal há uma guarda habilitada.
    guardas = z3.Or(
        pc == INIT,
        z3.And(pc == SKIP, z3.Or(y == 0,
            z3.And(y != 0, even), z3.And(y != 0, odd))),
        z3.And(pc == LEFT, z3.Or(of_l, z3.Not(of_l))),
        z3.And(pc == RIGHT, z3.Or(of_r, z3.Not(of_r)))
    )

    troca = tuple(zip(S + S1, S1 + S2))
    return {
        "n": n, "S": S, "S1": S1,
        "locais": dict(zip(("init", "skip", "left", "right", "stop", "error_L",
"error_R"),
                           (INIT, SKIP, LEFT, RIGHT, STOP, ERROR_L, ERROR_R))),
        "I": I, "T": T, "E": E, "I_c": I_c,
        "P": produto_modular(S), "J": J(S), "J1": J(S1),
        "J_c": J_c(S), "J_c1": J_c(S1),
        "look": lookahead(S, S1), "look12": lookahead(S1, S2),
        "T12": z3.substitute(T, *troca),
```

```
        "term": terminal(S), "guardas": guardas,
    }
```

2.3. Construção e primeira verificação

A função anterior devolve as fórmulas I, T, E, os estados simbólicos e os predicados auxiliares que serão explicados nas próximas secções. As oito ligações são construídas por uma função comum `ligacao`, que acrescenta a conservação dos operandos originais e a guarda de cada ramo.

Usamos $n = 8$ para as provas principais. Começamos por verificar se existe pelo menos uma transição válida. Um resultado `sat` confirma a consistência inicial da relação, mas **não prova por si só** a sua correção.

Nos restantes testes, `unsat` para a negação da propriedade significa que não foi encontrado um contraexemplo simbólico à condição testada.

In [ ]:
```
# Contribuição LLM: verificações SMT com resultados explícitos.
n = 8  # Largura utilizada nas provas do notebook.
M = construir_sfots(n)

def verificar(nome, formula, esperado):
    """Verifica uma obrigação SMT sem aceitar respostas inconclusivas."""
    solver = z3.Solver()
    # 10 s foram insuficientes para o passo indutivo de J no ambiente usado.
    # O limite é uma decisão operacional, não altera a fórmula verificada.
    solver.set(timeout=120000, random_seed=0)
    solver.add(formula)
    resultado = solver.check()
    print(f"{nome}: {resultado}")
    if resultado == z3.unknown:
        motivo = solver.reason_unknown()
        raise RuntimeError(
            f"{nome}: Z3 inconclusivo ({motivo}); "
            "não é uma prova SAT nem UNSAT."
        )
    if resultado != esperado:
        raise AssertionError(f"{nome}: esperado {esperado}; obtido {resultado}")
    return resultado

verificar("(a) Relação T satisfazível", M["T"], z3.sat)
```

```
(a) Relação T satisfazível: sat
```

sat

2.4. Testes dirigidos das transições

Antes de discutir as propriedades globais, fazemos alguns testes curtos que exercitam **ramos distintos** do CFA: escolha por paridade, paragem normal e precedência do overflow. Verificamos igualmente que uma transição proibida não pode ocorrer. Estes testes não substituem a k-indução.

In [ ]:
```
# Contribuição LLM: seis testes de ramos do CFA, incluindo casos proibidos.
def _testar_ramos_do_cfa():
    p, a, b, x, y, z = M["S"]
    p1, a1, b1, x1, y1, z1 = M["S1"]
    loc = M["locais"]
    T = M["T"]

    casos = [
        ("skip: y=0 chega a stop",
         z3.And(T, p == loc["skip"], y == 0, p1 == loc["stop"]), z3.sat),
        ("skip: par não nulo chega a left",
         z3.And(T, p == loc["skip"], y == 6, p1 == loc["left"]), z3.sat),
        ("skip: ímpar não pode chegar a left",
         z3.And(T, p == loc["skip"], y == 5, p1 == loc["left"]), z3.unsat),
        ("left: overflow não regressa a skip",
         z3.And(T, p == loc["left"], x == 128, p1 == loc["skip"]), z3.unsat),
        ("right: overflow chega a error_R",
         z3.And(T, p == loc["right"], x == 1, z == 255,
                p1 == loc["error_R"]), z3.sat),
        ("stop: não admite sucessores",
         z3.And(T, p == loc["stop"]), z3.unsat),
    ]
    for nome, formula, esperado in casos:
        verificar(nome, formula, esperado)
    print(f"{len(casos)} testes dirigidos concluídos.")
_testar_ramos_do_cfa()
```

```
skip: y=0 chega a stop: sat
skip: par não nulo chega a left: sat
skip: ímpar não pode chegar a left: unsat
left: overflow não regressa a skip: unsat
right: overflow chega a error_R: sat
stop: não admite sucessores: unsat
6 testes dirigidos concluídos.
```

3. Invariante da multiplicação — alínea 3.b

A igualdade indicada no enunciado é:

$$P(S) \equiv xy + z = ab.$$

3.1. Por que não é invariante no estado inicial?

O predicado inicial exige apenas $pc = init$. Como x, y, z ainda não foram atribuídos pelo algoritmo, existe um estado inicial com $\neg P$. Por isso, o caso base de indução de P é **satisfazível**.

Depois da inicialização, a igualdade é pretendida no sentido **modular**, pois todos os registos têm largura n. Definimos:

$$Q(S) \equiv (pc = init) \vee P(S).$$

3.2. Reforço indutivo

A propriedade Q, isoladamente, não é necessariamente 1-indutiva: o passo indutivo pode usar estados não alcançáveis com `pc=left` e y ímpar, incompatíveis com a guarda de entrada nesse nodo.

O predicado auxiliar C expressa as guardas necessárias:

$$C(S) \equiv (pc = left \Rightarrow y \neq 0 \wedge even(y)) \wedge (pc = right \Rightarrow y \neq 0 \wedge odd(y)).$$

Utilizamos o invariante reforçado $J = Q \wedge C$, comprovando:

$$I(S) \wedge \neg J(S), \qquad J(S) \wedge T(S, S') \wedge \neg J(S').$$

Se ambas as consultas forem `unsat`, J é 1-indutivo, isto é, k-indução com $k = 1$. Como $J \Rightarrow Q$, obtém-se a igualdade modular após a ação `init`. **Não se conclui** que P é um invariante global válido antes da inicialização.

In [ ]:
```
# Contribuição LLM: implementação da prova da alínea (b).
verificar("(b) Contraexemplo a P no início",
          z3.And(M["I"], z3.Not(M["P"])), z3.sat)
verificar("(b) Base de J",
          z3.And(M["I"], z3.Not(M["J"])), z3.unsat)
verificar("(b) Passo de J",
          z3.And(M["J"], M["T"], z3.Not(M["J1"])), z3.unsat)
```

```
(b) Contraexemplo a P no início: sat
(b) Base de J: unsat
(b) Passo de J: unsat
```

unsat

4. Segurança — alínea 3.c

Acrescentamos ao estado inicial as restrições do enunciado:

$$I_c(S) \equiv I(S) \wedge (a < 2^{n/2}) \wedge (b < 2^{n/2}).$$

Para evitar arredondamentos com n ímpar, o maior inteiro permitido é $\lfloor \sqrt{2^n - 1} \rfloor$. As comparações são sem sinal (`ULE`). Destas condições resulta $ab < 2^n$.

4.1. A igualdade modular não basta

A igualdade em BitVecs de n bits pode continuar verdadeira mesmo quando ocorreu overflow. Por isso, para provar a segurança, calculamos a igualdade $xy + z = ab$ em BitVecs de $2n$ **bits**, evitando truncatura na multiplicação dos valores de n bits.

Definimos:

$$J_c(S) \equiv B_c(S) \wedge (pc = init \vee P_{exata}(S)) \wedge C(S) \wedge \neg E(S).$$

A propriedade pretendida é $G\neg E$. Demonstramos por 1-indução que J_c é invariante com o estado inicial restringido:

$$I_c(S) \wedge \neg J_c(S), \qquad J_c(S) \wedge T(S,S') \wedge \neg J_c(S').$$

A primeira consulta confirma ainda que I_c admite estados: uma prova de segurança baseada num estado inicial impossível seria vazia.

In [ ]:
```
# Contribuição LLM: implementação da prova da alínea (c).
verificar("(c) Estado inicial restringido existe", M["I_c"], z3.sat)
verificar("(c) Base de J_c",
          z3.And(M["I_c"], z3.Not(M["J_c"])), z3.unsat)
verificar("(c) Passo de J_c",
          z3.And(M["J_c"], M["T"], z3.Not(M["J_c1"])), z3.unsat)
```

```
(c) Estado inicial restringido existe: sat
(c) Base de J_c: unsat
(c) Passo de J_c: unsat
```

unsat

5. Terminação — alínea 3.d

5.1. Propriedade de animação

Não basta demonstrar a ausência de erro para concluir que o programa termina. Na alínea 3.d os operandos iniciais **não são restringidos**. A execução pode terminar em `stop`, `error_L` ou `error_R`.

$$Term(S) \equiv (pc = stop) \vee E(S), \qquad F\,Term.$$

Interpreta-se $F\,Term$ sobre execuções máximas: uma execução que chega a um terminal é finita e termina nesse estado. Não acrescentamos transições artificiais de permanência nos terminais.

5.2. Função de progresso

Seja a função de valores naturais:

$$R(S) = \begin{cases} 2b+2 & pc = init \\ 2y+1 & pc = skip \\ 2y & pc = left \text{ ou } pc = right \\ 0 & Term(S). \end{cases}$$

A função é codificada em $n+2$ bits, evitando overflow do próprio contador de progresso. Entre estados alcançáveis não terminais ela diminui estritamente: em `skip` há uma descida para o ramo escolhido; em `right` diminui y; em `left` y é par positivo e é dividido por 2.

5.3. Lookahead e cobertura de guardas

O predicado $P^*(S, S')$ exige a descida estrita de R entre estados não terminais, juntamente com $C(S)$ e $C(S')$. A prova verifica um caso base e um passo indutivo em três cópias S_0, S_1, S_2:

$$I(S_0) \wedge T_{01} \wedge \neg P^*_{01},$$

$$P^*_{01} \wedge T_{01} \wedge T_{12} \wedge \neg P^*_{12}.$$

Verificamos também a cobertura das guardas nos nodos não terminais. O decréscimo numa ordem bem fundada e a ausência de bloqueios não terminais implicam que todas as execuções máximas terminam.

In [ ]:
```
# Contribuição LLM: verificação do lookahead e das guardas da alínea (d).
verificar("(d) Base do lookahead",
          z3.And(M["I"], M["T"], z3.Not(M["look"])), z3.unsat)
verificar("(d) Passo do lookahead",
          z3.And(M["look"], M["T"], M["T12"], z3.Not(M["look12"])),
          z3.unsat)
verificar("(d) Sem bloqueios não terminais (guardas)",
          z3.And(z3.Not(M["term"]), z3.Not(M["guardas"])), z3.unsat)
```

```
(d) Base do lookahead: unsat
(d) Passo do lookahead: unsat
(d) Sem bloqueios não terminais (guardas): unsat
```

unsat

6. Validação por percursos concretos

As provas SMT são acompanhadas por exemplos específicos, como nos outros trabalhos. Escolhemos apenas casos com funções diferentes:

Caso	Operandos	Resultado pretendido
Produto normal	3×5	`stop` , $z = 15$
Multiplicador nulo	7×0	`stop` , $z = 0$
Multiplicando nulo	0×7	`stop` , $z = 0$
Overflow na duplicação	64×4	`error_L`
Overflow na soma	86×3	`error_R`
Fronteira segura	15×15	`stop` , $z = 225$

Um pequeno intérprete em Python constrói os estados esperados **sem Z3**. Em seguida, cada par consecutivo é confirmado com a fórmula T. Esta é uma validação independente da fórmula, mas não uma prova universal. O comportamento dos ramos proibidos foi testado separadamente na secção 2.

In [ ]:

```python
# Contribuição LLM: testes com oráculo Python independente e conferência Z3.
def percurso_python(a0, b0, largura):
    """Calcula o percurso esperado apenas pelas ações do CFA em Python."""
    capacidade = 1 << largura
    if not (0 <= a0 < capacidade and 0 <= b0 < capacidade):
        raise ValueError("Operandos fora da representação sem sinal")

    estado = dict(pc="init", a=a0, b=b0, x=0, y=0, z=0)
    percurso = [estado]
    for _ in range(4 * largura + 5):
        seguinte = estado.copy()
        local = estado["pc"]
        if local == "init":
            seguinte.update(pc="skip", x=a0, y=b0, z=0)
        elif local == "skip":
            y0 = estado["y"]
            seguinte["pc"] = "stop" if y0 == 0 else (
                "left" if y0 % 2 == 0 else "right"
            )
        elif local == "left":
            dobro = 2 * estado["x"]
            seguinte.update(
                pc="error_L" if dobro >= capacidade else "skip",
                x=dobro % capacidade, y=estado["y"] // 2
            )
        elif local == "right":
            soma = estado["z"] + estado["x"]
            seguinte.update(
                pc="error_R" if soma >= capacidade else "skip",
                y=estado["y"] - 1, z=soma % capacidade
            )
        else:
            return percurso
        percurso.append(seguinte)
        estado = seguinte
    raise AssertionError("O percurso excedeu o limite defensivo")

def _testar_percursos_concretos():
    # Casos distintos; não se repetem verificações equivalentes.
    exemplos = [
        (3, 5, "stop", 15),
        (7, 0, "stop", 0),
        (0, 7, "stop", 0),
        (64, 4, "error_L", 0),
        (86, 3, "error_R", 2),
        (15, 15, "stop", 225),
    ]
    campos = ("pc", "a", "b", "x", "y", "z")
    total_transicoes = 0

    for a0, b0, destino, z_esperado in exemplos:
        percurso = percurso_python(a0, b0, n)
        final = percurso[-1]
        assert (final["pc"], final["z"]) == (destino, z_esperado)
```

```
        for atual, seguinte in zip(percurso, percurso[1:]):
            s = z3.Solver()
            s.set(timeout=5000, random_seed=0)
            s.add(M["T"])
            for simbolo, chave in zip(M["S"], campos):
                valor = atual[chave]
                s.add(simbolo == (M["locais"][valor] if chave == "pc"
                                  else z3.BitVecVal(valor, n)))
            for simbolo, chave in zip(M["S1"], campos):
                valor = seguinte[chave]
                s.add(simbolo == (M["locais"][valor] if chave == "pc"
                                  else z3.BitVecVal(valor, n)))
            assert s.check() == z3.sat, (
                f"Transição inválida para {a0} x {b0}: "
                f"{atual['pc']} → {seguinte['pc']}"
            )
            total_transicoes += 1

        print(f"{a0} × {b0}: {final['pc']}; z={final['z']}; "
              f"{len(percurso)-1} transições validadas")

    assert total_transicoes > 0
    print(f"Resultado: {len(exemplos)} exemplos, {total_transicoes} "
          "transições confrontadas com o SFOTS.")
_testar_percursos_concretos()
```

```
3 × 5: stop; z=15; 10 transições validadas
7 × 0: stop; z=0; 2 transições validadas
0 × 7: stop; z=0; 12 transições validadas
64 × 4: error_L; z=0; 5 transições validadas
86 × 3: error_R; z=2; 7 transições validadas
15 × 15: stop; z=225; 16 transições validadas
Resultado: 6 exemplos, 52 transições confrontadas com o SFOTS.
```

6.1. Conferência de percursos extraídos do Z3

Para não depender apenas dos percursos construídos em Python, extraímos três execuções com o próprio Z3 e comparamos **todos os estados** com os estados esperados pelo intérprete independente. Estes casos incluem terminação normal e os dois tipos de erro.

In [ ]:
```
# Contribuição LLM: extração de percursos Z3 e comparação independente.
def _testar_percursos_z3():
    campos = ("pc", "a", "b", "x", "y", "z")
    terminais = {"stop", "error_L", "error_R"}

    def executar_z3(a0, b0):
        # Em init, os valores iniciais de x,y,z podem ser arbitrários.
        atual = {"pc": "init", "a": a0, "b": b0,
                 "x": 0, "y": 0, "z": 0}
        percurso = [atual.copy()]
        for _ in range(4 * n + 5):
            if atual["pc"] in terminais:
                return percurso
            solver = z3.Solver()
            solver.set(timeout=5000, random_seed=0)
```

```
            solver.add(M["T"])
            for simbolo, chave in zip(M["S"], campos):
                valor = atual[chave]
                solver.add(simbolo == (
                    M["locais"][valor] if chave == "pc"
                    else z3.BitVecVal(valor, n)))
            assert solver.check() == z3.sat, "Sem sucessor válido"
            solucao = solver.model()
            seguinte = {}
            for simbolo, chave in zip(M["S1"], campos):
                valor = solucao.eval(simbolo, model_completion=True)
                seguinte[chave] = (valor.decl().name() if chave == "pc"
                                   else valor.as_long())
            percurso.append(seguinte)
            atual = seguinte
        raise AssertionError("Foi ultrapassado o limite de passos")

    for aa, bb in ((3, 5), (64, 4), (86, 3)):
        obtido = executar_z3(aa, bb)
        esperado = percurso_python(aa, bb, n)
        assert obtido == esperado, f"Percurso Z3 diferente do Python: {aa}x{bb}"
        print(f"Percurso extraído e validado: {aa} × {bb}, "
              f"{len(obtido)-1} transições, final={obtido[-1]['pc']}")
_testar_percursos_z3()
```

```
Percurso extraído e validado: 3 × 5, 10 transições, final=stop
Percurso extraído e validado: 64 × 4, 5 transições, final=error_L
Percurso extraído e validado: 86 × 3, 7 transições, final=error_R
```

7. Eficiência computacional e escalabilidade

Para cumprir a exigência das regras gerais sobre eficiência e escala, medimos **uma obrigação representativa**, o passo indutivo de segurança, para $n \in \{4, 8, 12\}$, com três repetições por largura.

Reutilizamos a mesma função `construir_sfots(n)` da alínea 3.a: não existe outro CFA criado apenas para os testes de desempenho. Cada repetição cria um solver novo, com semente `0` e limite de 5000 ms para `check()`. O tempo medido com `perf_counter()` inclui a construção das fórmulas e a resolução. O limite de 5000 ms restringe apenas o solver, não a construção em Python.

Os resultados são impressos diretamente após a execução, sem números pré-preenchidos. `unknown` é inconclusivo e não pode ser tratado como `unsat`. Três repetições são adequadas a uma **comparação exploratória**, mas não a estimar a complexidade assintótica do algoritmo.

Resultados observados na execução de 31/10/2026

Os valores seguintes foram recolhidos na execução do notebook no computador do grupo e comunicados através das capturas de ecrã. São resultados dessa execução, não previsões

nem tempos fixos de futuras execuções.

Largura	Estados das três consultas	Tempos (ms)	Mediana (ms)
4 bits	`unsat` , `unsat` , `unsat`	26,12; 27,24; 23,44	26,12
8 bits	`unsat` , `unsat` , `unsat`	1843,96; 2127,10; 1870,38	1870,38
12 bits	`unknown` , `unknown` , `unknown`	5023,50; 5011,41; 5013,38	5013,38

As três consultas de 4 e de 8 bits foram concluídas com `unsat` : não foi encontrado contraexemplo ao passo indutivo de segurança nessas larguras. As três consultas de 12 bits foram **inconclusivas**, compatíveis com o limite de 5 segundos imposto a `solver.check()` . Como não foi registado `reason_unknown()` nesta execução, não atribuímos formalmente a causa do `unknown` ao timeout; apenas observamos que o tempo foi próximo desse limite. Em particular, `unknown` **não significa** `sat` **nem** `unsat` .

Estes dados mostram que a verificação foi mais demorada na instância de 8 bits do que na de 4 bits e não terminou no limite de 5 segundos para 12 bits. Não permitem concluir uma lei geral de escalabilidade, nem alteram as provas principais realizadas com 8 bits.

In [ ]:
```python
# Contribuição LLM: experiência exploratória curta e reproduzível.
def _avaliar_desempenho():
    from statistics import median

    for largura in (4, 8, 12):
        tempos = []
        resultados = []
        for _ in range(3):
            inicio = perf_counter()
            modelo = construir_sfots(largura)
            solver = z3.Solver()
            solver.set(timeout=5000, random_seed=0)
            solver.add(modelo["J_c"], modelo["T"], z3.Not(modelo["J_c1"]))
            resultado = solver.check()
            duracao = 1000 * (perf_counter() - inicio)
            tempos.append(duracao)
            resultados.append(str(resultado))
        print(f"n={largura:2d} | resultados={resultados} | "
              f"tempos(ms)={[round(t, 2) for t in tempos]} | "
              f"mediana={median(tempos):.2f} ms")
        if set(resultados) != {"unsat"}:
            print("  Atenção: existe um resultado inconclusivo ou inesperado.")
_avaliar_desempenho()
```

```
n= 4 | resultados=['unsat', 'unsat', 'unsat'] | tempos(ms)=[26.12, 27.24, 23.44] | m
ediana=26.12 ms
n= 8 | resultados=['unsat', 'unsat', 'unsat'] | tempos(ms)=[1843.96, 2127.1, 1870.3
8] | mediana=1870.38 ms
n=12 | resultados=['unknown', 'unknown', 'unknown'] | tempos(ms)=[5023.5, 5011.41, 5
013.38] | mediana=5013.38 ms
  Atenção: existe um resultado inconclusivo ou inesperado.
```

8. Síntese dos requisitos e limitações

Alínea	Fórmula ou componente	Evidência de verificação
3.a	I, $T = \bigvee T_i$, E	`sat` de T, seis testes dirigidos de ramos e percursos completos.
3.b	P, Q, $J = Q \wedge C$	Contraexemplo inicial a P; base e passo de J.
3.c	I_c, J_c	Estado inicial satisfazível; base e passo indutivo.
3.d	$FTerm$, função R, lookahead	Base, passo indutivo e cobertura das guardas.
Regras gerais	Testes e desempenho	Exemplos normais/de erro, validação independente e medições em três larguras.

Limitações

- **Sem sinal:** a deteção de overflow foi definida para valores não negativos sem sinal. O material de apoio também apresenta a interpretação com sinal.
- **Largura finita:** as provas principais são executadas para $n = 8$. Executar para outras larguras não equivale a provar todas simultaneamente.
- **Inicialização:** o invariante matemático só é exigido após a ação `init`.
- **Terminação:** os estados `stop`, `error_L`, `error_R` são terminais; terminar por overflow não significa concluir a multiplicação corretamente.
- **Desempenho:** para 12 bits, as três medições terminaram em `unknown` junto ao limite de 5 segundos. Os tempos correspondem a consultas inconclusivas e não ao tempo de uma demonstração concluída. Não é legítimo extrapolar a tendência para larguras maiores.

9. Reprodutibilidade

O notebook foi preparado para ser executado em **Python 3.14.6**, **Marimo 0.25.0** e **Z3 5.1.0**, versões comunicadas no desenvolvimento. É necessário confirmar que coincidem com as versões efetivas do ambiente de entrega. O cabeçalho declara as dependências `marimo` e `z3-solver`.

Para reproduzir:

1. Colocar o ficheiro numa pasta de trabalho e instalar `marimo` e `z3-solver`.
2. Abrir com `marimo edit multiplicacao.py`.
3. Executar todas as células, com $n = 8$ para as provas principais.
4. Confirmar os estados `sat` / `unsat`, a validação dos percursos e as medições impressas pela célula de desempenho.

5. Exportar para PDF **depois** da execução, garantindo que os resultados no PDF correspondem à última versão do notebook.

O código não utiliza aleatoriedade para escolher os operandos dos testes. O Z3 recebe a semente `0` nas consultas controladas. As provas usam um limite de 120 segundos; `unknown` permanece inconclusivo. A tabela da secção 7 regista uma execução concreta; numa nova execução devem ser consultados também os resultados atualizados no output.

In [ ]:
```python
# Diagnóstico do ambiente; os dados apresentados são obtidos nesta execução.
import sys
import marimo as marimo_ambiente
import z3 as z3_ambiente
print("Python:", sys.version.split()[0])
print("Marimo:", marimo_ambiente.__version__)
print("Z3:", z3_ambiente.get_version_string())
```

```
Python: 3.14.6
Marimo: 0.25.0
Z3: 5.1.0
```

10. Registo da utilização de ferramentas LLM

Foi utilizada a ferramenta ChatGPT como apoio à interpretação e implementação. As sugestões técnicas estão identificadas no código através de comentários. Os resultados SMT e as medições do notebook **só podem ser atribuídos à execução real pelos alunos**, após confirmação no Marimo.

Componente	Apoio da LLM	Verificação necessária pelo grupo
Interpretação do CFA	Organização das variáveis e oito ligações.	Confirmar o mapeamento com o diagrama do professor.
SFOTS e overflow	Proposta das fórmulas e da implementação Z3.	Validar os ramos e a interpretação sem sinal.
Invariante 3.b	Identificação da inicialização e reforço com C.	Explicar caso base e passo indutivo.
Segurança 3.c	Construção de I_c e J_c.	Confirmar resultados e o uso dos $2n$ bits.
Terminação 3.d	Função de progresso e lookahead.	Justificar a descida estrita e os terminais.
Testes e desempenho	Exemplos, validação independente e medição.	Executar e interpretar resultados reais.
Documentação	Organização e redação do notebook.	Rever e assumir as decisões efetivamente adotadas.

Diálogo de desenvolvimento: https://chatgpt.com/share/6ae4ee04-d360-83ed-970f-1595d13ee104

Referências

1. *Lógica Computacional 2026–2027 — Trabalhos Práticos*, Exercício 3 e regras gerais dos trabalhos.
2. *Lógica Computacional: Multiplicação de Inteiros*, CFA e operações com BitVecs.
3. Capítulos 3 e 4 da unidade curricular, k-indução, sistemas de transição e propriedades de segurança/animação.
4. *Uso de Ferramentas LLM*, orientações da disciplina.

11. Conclusão

O SFOTS representa os sete nodos e as oito ligações do CFA. A propriedade $xy + z = ab$ não é global no estado inicial, mas é estabelecida após `init`. Sob as restrições da alínea 3.c, a indução do invariante reforçado prova a inacessibilidade dos erros na interpretação sem sinal. A função de progresso e o lookahead demonstram terminação normal ou por erro para quaisquer operandos iniciais. A avaliação é acompanhada por testes concretos distintos e uma experiência exploratória de desempenho. Nesta experiência, o passo de segurança foi `unsat` para 4 e 8 bits e inconclusivo para 12 bits com o limite de 5 segundos; esse limite não invalida as provas com 8 bits.